

目录

×

01、什么是 ReAct 模式？

[它和 Chain-of-Thought 有什...](#)

02、Agent 怎么知道该调用哪...

[如果 LLM 返回了不存在的工...](#)

03、ReAct 循环会死循环吗？

[常见的死循环场景](#)

[PaiCLI 怎么处理死循环的？](#)

04、什么是 Plan-and-Execut...

[它比 ReAct 好在哪？](#)

05、Plan-and-Execute 里的...

[某个任务失败了怎么办](#)

06、Multi-Agent 协作是怎么...

[各角色的 system prompt 有...](#)

07、Reviewer 审查不通过怎...

[这个模式和 Code Review 有...](#)

08、同一轮 LLM 返回多个 to...

[并行执行的性能提升有多大](#)

09、并行工具调用会有冲突吗

10、Token 预算是怎么管理的？

11、ReAct、Plan-and-Execu...

12、如果让你从零设计一个 A...

13、面试中怎么介绍你的 Age...

ending

目录

01、什么是 ReAct 模式？

它和 Chain-of-Thought 有什...

02、Agent 怎么知道该调用哪...

如果 LLM 返回了不存在的工...

03、ReAct 循环会死循环吗？

常见的死循环场景

PaiCLI 怎么处理死循环的？

04、什么是 Plan-and-Execut...

它比 ReAct 好在哪？

05、Plan-and-Execute 里的...

某个任务失败了怎么办

06、Multi-Agent 协作是怎么...

各角色的 system prompt 有...

07、Reviewer 审查不通过怎...

这个模式和 Code Review 有...

08、同一轮 LLM 返回多个 to...

并行执行的性能提升有多大

09、并行工具调用会有冲突吗

10、Token 预算是怎么管理的？

11、ReAct、Plan-and-Execu...

12、如果让你从零设计一个 A...

13、面试中怎么介绍你的 Age...

ending

https://paicoding.com/column/17/15

2/17

Search

AI Agent 面试题第一弹：ReAct、Plan-and-Execute、Multi-Agent 核心架构 13 题

原创

第一弹，聚焦 **Agent 核心架构**——ReAct、Plan-and-Execute、Multi-Agent、异步并行。

这几个方向面试出现的频率最高，也是 PaiCLI 第 1、2、5、7 期的核心内容。

01、什么是 ReAct 模式？

ReAct 是 Reasoning + Acting 的缩写，Yao et al.（姚顺雨）在 2022 年提出。

核心就一句话：让 LLM 在推理的同时能执行动作，根据动作结果继续推理，形成一个闭合的循环。

目录

[01、什么是 ReAct 模式？](#)

[它和 Chain-of-Thought 有什...](#)

[02、Agent 怎么知道该调用哪...](#)

[如果 LLM 返回了不存在的工...](#)

[03、ReAct 循环会死循环吗？](#)

[常见的死循环场景](#)

[PaiCLI 怎么处理死循环的？](#)

[04、什么是 Plan-and-Execut...](#)

[它比 ReAct 好在哪？](#)

[05、Plan-and-Execute 里的...](#)

[某个任务失败了怎么办](#)

[06、Multi-Agent 协作是怎么...](#)

[各角色的 system prompt 有...](#)

[07、Reviewer 审查不通过怎...](#)

[这个模式和 Code Review 有...](#)

[08、同一轮 LLM 返回多个 to...](#)

[并行执行的性能提升有多大](#)

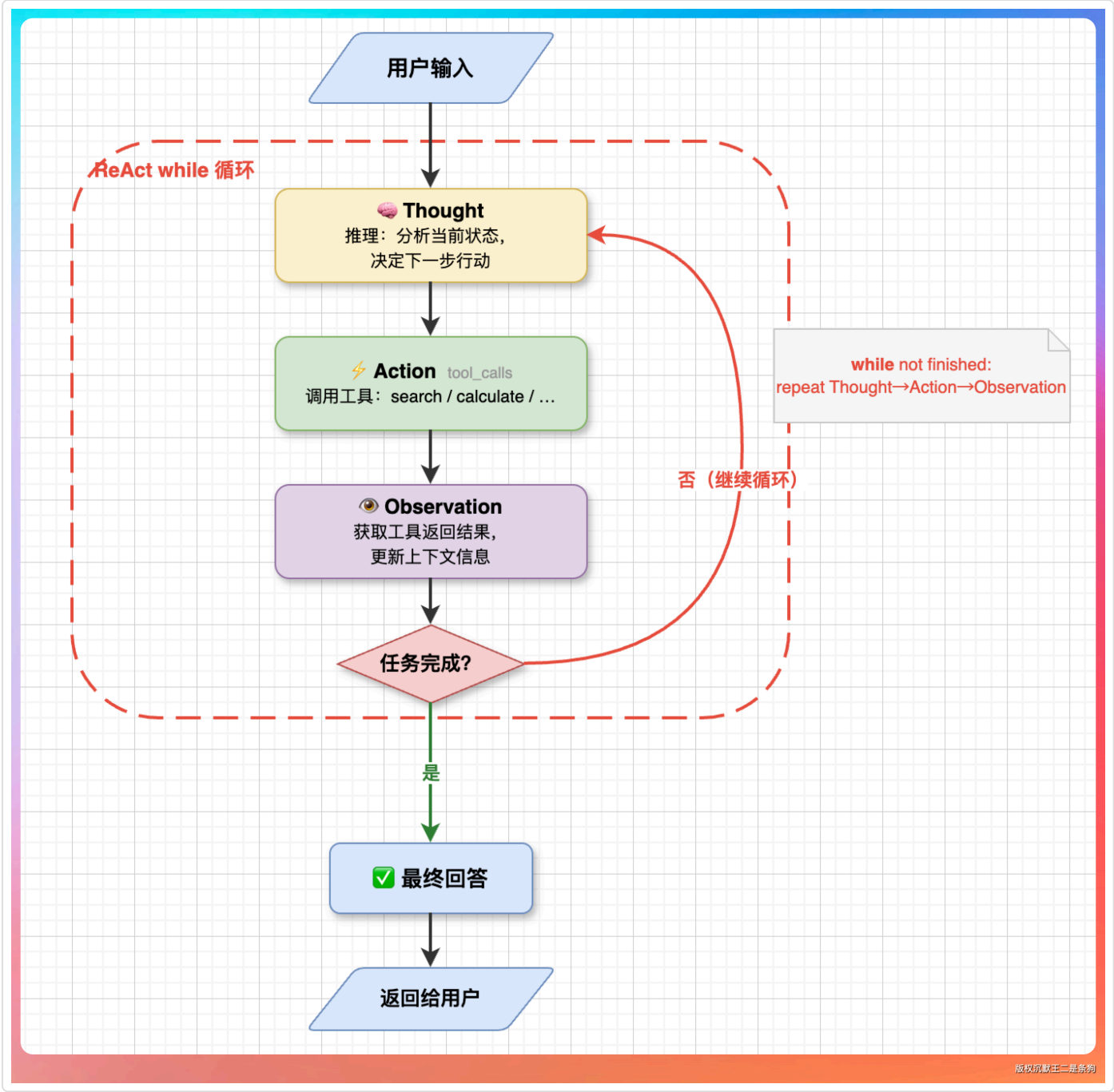
[09、并行工具调用会有冲突吗](#)

[10、Token 预算是怎么管理的？](#)

[11、ReAct、Plan-and-Execu...](#)

[12、如果让你从零设计一个 A...](#)

[13、面试中怎么介绍你的 Age... ending](#)



PaiCLI 第一期的 `Agent.java` 就是一个标准的 ReAct 实现。核心是一个 while 循环，每轮做三件事：

- 把消息历史发给 LLM、
- 检查响应里有没有 `tool_calls`
- 有的话执行工具把结果塞回历史。

LLM 不再返回 `tool_calls` 就退出循环，把最终回复输出给用户。

整个 Agent 的骨架就这么简单。

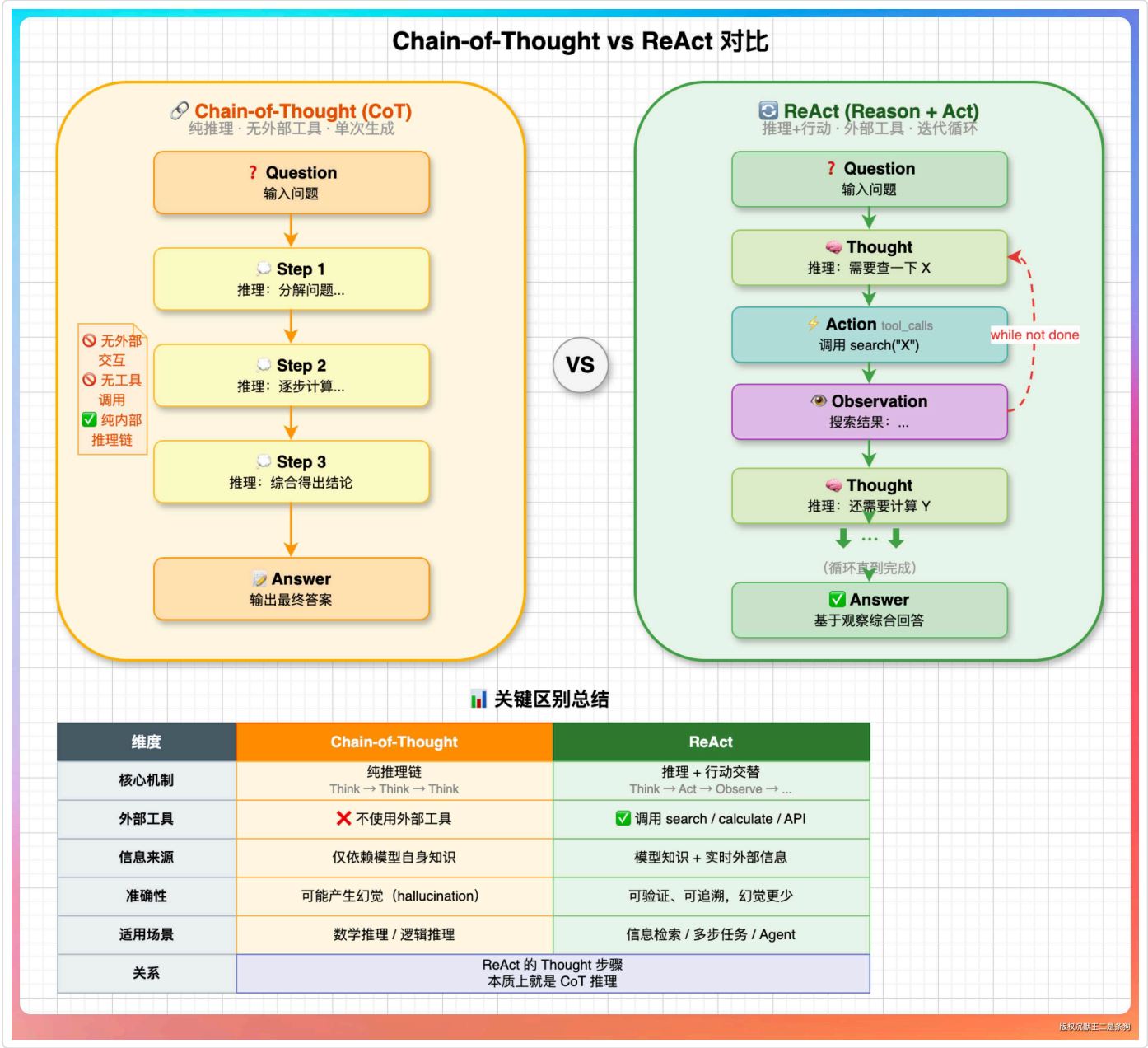
它和 Chain-of-Thought 有什么区别？

Chain-of-Thought (CoT) 只推理不执行。

LLM 一口气想完所有步骤，直接输出最终答案。做数学题、逻辑推理可以，但碰到“帮我读一下 pom.xml”这种需要外部信息的任务就歇菜了——LLM 没有读文件的能力，想得再好也是瞎猜。

目录

- [01、什么是 ReAct 模式？](#)
[它和 Chain-of-Thought 有什...](#)
- [02、Agent 怎么知道该调用哪...](#)
[如果 LLM 返回了不存在的工...](#)
- [03、ReAct 循环会死循环吗？](#)
[常见的死循环场景](#)
[PaiCLI 怎么处理死循环的？](#)
- [04、什么是 Plan-and-Execut...](#)
[它比 ReAct 好在哪？](#)
- [05、Plan-and-Execute 里的...](#)
[某个任务失败了怎么办](#)
- [06、Multi-Agent 协作是怎么...](#)
[各角色的 system prompt 有...](#)
- [07、Reviewer 审查不通过怎...](#)
[这个模式和 Code Review 有...](#)
- [08、同一轮 LLM 返回多个 to...](#)
[并行执行的性能提升有多大](#)
- [09、并行工具调用会有冲突吗](#)
- [10、Token 预算是怎么管理的？](#)
- [11、ReAct、Plan-and-Execu...](#)
- [12、如果让你从零设计一个 A...](#)
- [13、面试中怎么介绍你的 Age...](#)
[ending](#)



ReAct 的突破在于加了 Action 和 Observation 两个环节。LLM 想到“我需要读 pom.xml”，就输出一个 `read_file` 的 `tool_call`，Agent 真去读了文件，把内容返回回来，LLM 基于真实的内容继续推理。

用一个表格说清楚两者的边界：

维度	CoT	ReAct
能力范围	纯推理	推理 + 外部工具调用
信息来源	训练数据里的知识	实时获取（文件、命令、搜索）
适合场景	数学、逻辑、代码生成	需要与外部世界交互的任务
典型产品	ChatGPT 的思考过程	Claude Code、PaiCLI、Cursor

面试官追问到这一步，可以补一句：

PaiCLI 的 LLM 响应里也有 `reasoning_content`（思考过程），这个其实就是 CoT 的部分。

ReAct 不是替代 CoT，而是在 CoT 的基础上加了行动能力。PaiCLI 的源码里，`reasoning_content` 只写日志不进下一轮对话历史，避免思考过程占用 Token 预算。

02、Agent 怎么知道该调用哪个工具？

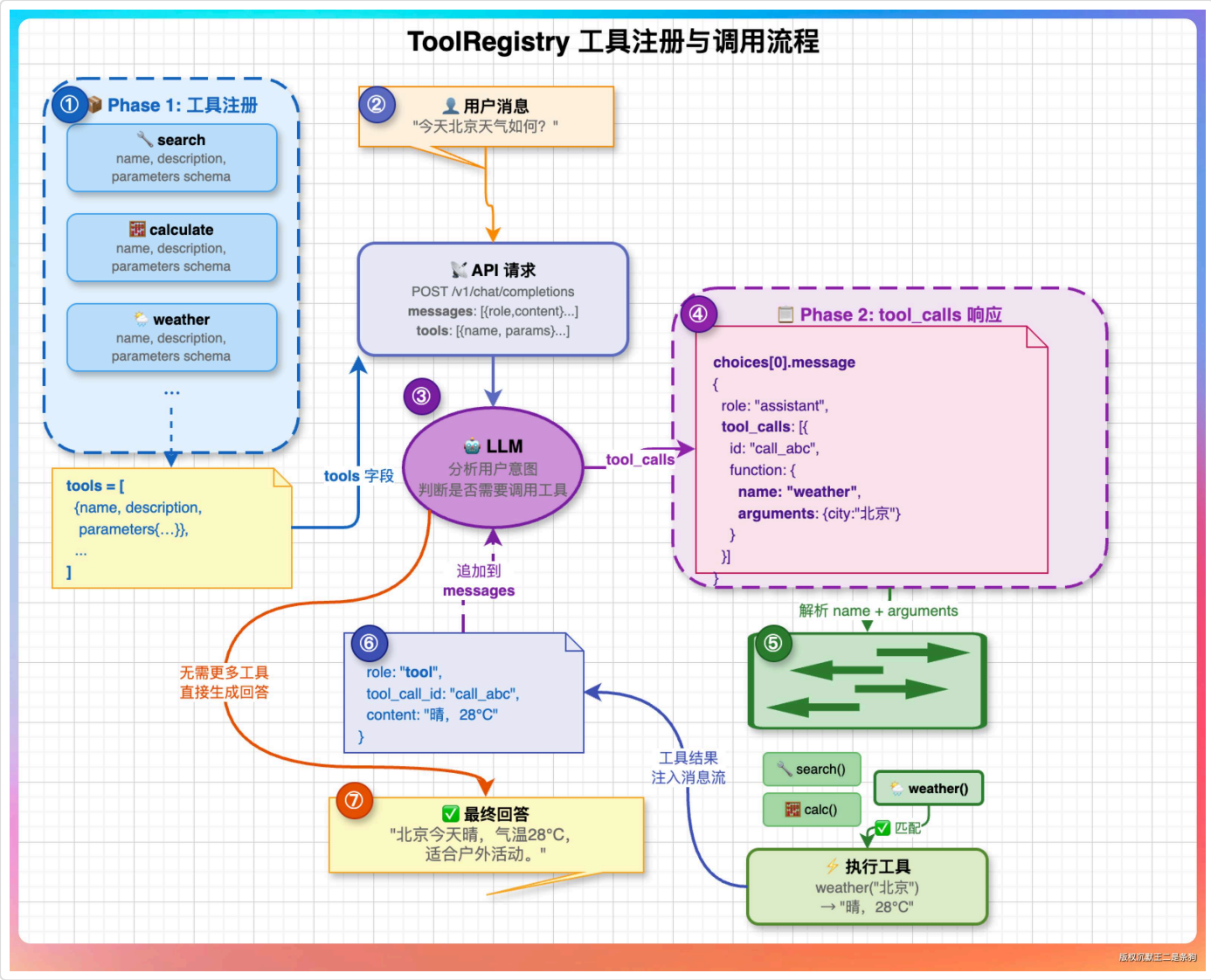
这道题很多人会答错，以为 Agent 里有个什么路由规则在做工具匹配。实际上 **Agent 本身不做工具选择，选择权完全在 LLM 手里**。

流程是这样的：Agent 在构造请求时，把所有可用工具的定义（名称 + 描述 + 参数 JSON Schema）放在请求体的 `tools` 字段里发给 LLM。LLM 根据用户意图和工具描述，在响应的 `tool_calls` 字段里返回工具名和参数 JSON。

这就是 OpenAI 定义的 **Function Calling** 协议，GLM、DeepSeek、Kimi 这些国产模型也都兼容。

目录

- 01、什么是 ReAct 模式？
它和 Chain-of-Thought 有什...
- 02、Agent 怎么知道该调用哪...
如果 LLM 返回了不存在的工...
- 03、ReAct 循环会死循环吗？
常见的死循环场景
PaiCLI 怎么处理死循环的？
- 04、什么是 Plan-and-Execut...
它比 ReAct 好在哪？
- 05、Plan-and-Execute 里的...
某个任务失败了怎么办
- 06、Multi-Agent 协作是怎么...
各角色的 `system_prompt` 有...
- 07、Reviewer 审查不通过怎...
这个模式和 Code Review 有...
- 08、同一轮 LLM 返回多个 to...
并行执行的性能提升有多大
- 09、并行工具调用会有冲突吗
- 10、Token 预算是怎么管理的？
- 11、ReAct、Plan-and-Execu...
- 12、如果让你从零设计一个 A...
- 13、面试中怎么介绍你的 Age...
ending



PaiCLI 的 `ToolRegistry.java` 维护了一个工具注册表。每个工具注册时提供 `name`、`description`、`parameters schema`。Agent 每次请求 LLM 前，从注册表拉出全量工具定义塞进请求体。LLM 返回 `tool_calls: [{name: "read_file", arguments: {path: "pom.xml"}}]`，Agent 就从注册表里找到 `read_file` 的执行逻辑来跑。

java 复制代码

```
// ToolRegistry.java 核心结构
private final Map<String, ToolDefinition> tools = new LinkedHashMap<>();
private final Map<String, ToolExecutor> executors = new LinkedHashMap<>();

public String executeTool(String name, String argumentsJson) {
    ToolExecutor executor = executors.get(name);
    if (executor == null) {
        return "未知工具: " + name;
    }
    return executor.execute(argumentsJson);
}
```

这里有个实战经验值得提一下：**工具描述的质量直接决定 LLM 的选择准确率。**

PaiCLI 早期 `execute_command` 的描述写得太简洁，LLM 经常用 `cat` 代替 `read_file` 读文件。后来在描述里加了“在项目根目录执行的短时 Shell 命令，如 `ls`、`mvn compile`，不要用来读取文件内容”，准确率就上去了。

如果 LLM 返回了不存在的工具名怎么办

`ToolRegistry.executeTool()` 做了兜底——找不到工具就返回 **“未知工具: xxx”**。这个错误信息作为 tool message 塞回对话历史，LLM 下一轮看到了会自动修正。

但如果 LLM 反复返回不存在的工具名，说明 `system prompt` 或工具描述有问题，需要优化 `prompt` 而不是加更多兜底逻辑。

03、ReAct 循环会死循环吗？

会。

常见的死循环场景

场景一：`execute_command` 执行失败，LLM 不甘心，换个参数再试，又失败，无限重试。比如让 Agent 编译项目，`mvn compile` 报错了，LLM 改了一下代码再编译，又报错，改了再编译……

目录

- 01、什么是 ReAct 模式？
 - 它和 Chain-of-Thought 有什...
- 02、Agent 怎么知道该调用哪...
 - 如果 LLM 返回了不存在的工...
- 03、ReAct 循环会死循环吗？
 - 常见的死循环场景
 - PaiCLI 怎么处理死循环的？
- 04、什么是 Plan-and-Execut...
 - 它比 ReAct 好在哪？
- 05、Plan-and-Execute 里的...
 - 某个任务失败了怎么办
- 06、Multi-Agent 协作是怎么...
 - 各角色的 `system prompt` 有...
- 07、Reviewer 审查不通过怎...
 - 这个模式和 Code Review 有...
- 08、同一轮 LLM 返回多个 to...
 - 并行执行的性能提升有多大
- 09、并行工具调用会有冲突吗
- 10、Token 预算是怎么管理的？
- 11、ReAct、Plan-and-Execu...
- 12、如果让你从零设计一个 A...
- 13、面试中怎么介绍你的 Age...ending

场景二：LLM 输出一段推理但不调用工具也不给最终答案。Agent 把这段推理塞回去再请求 LLM，LLM 继续自言自语，永远不收尾。

PaiCLI 怎么处理死循环的？

PaiCLI 源码里有四层防护：

第一层是 **Token 预算**。AgentBudget 根据当前模型的 maxContextWindow() 动态计算预算（默认取窗口的 80%），对话历史接近预算就触发摘要压缩或强制终止。



第二层是 **工具执行超时**。execute_command 有 60 秒超时，超时直接返回超时结果给 LLM，不会卡在那里。

第三层是 **用户取消**。运行中按 ESC 或输入 /cancel 可以请求取消当前 Agent run。ReAct、Plan、Team 三条路径在边界处都会检查取消信号。

第四层是 **摘要压缩兜底**。ContextCompressor 在对话历史膨胀到临界点时介入，把早期对话压缩成摘要释放空间。但如果压缩速度追不上膨胀速度（工具结果太大），最终还是会触发预算上限终止。

面试时说到这四层，面试官通常会追问“哪层最关键”。答案是 Token 预算——它是唯一一个和上下文窗口直接挂钩的约束。超时只管单个工具，用户取消依赖人的反应速度，摘要压缩有延迟。

04、什么是 Plan-and-Execute 模式？

Plan-and-Execute 是先规划后执行的两阶段模式。

用户输入一个复杂任务，Agent 不急着动手，先让 LLM 拆解成多个子任务并明确依赖关系，生成一份执行计划。用户确认后，再按计划逐个执行子任务。

PaiCLI 第 2 期实现了 PlanExecuteAgent.java，通过 /plan 命令触发。

目录

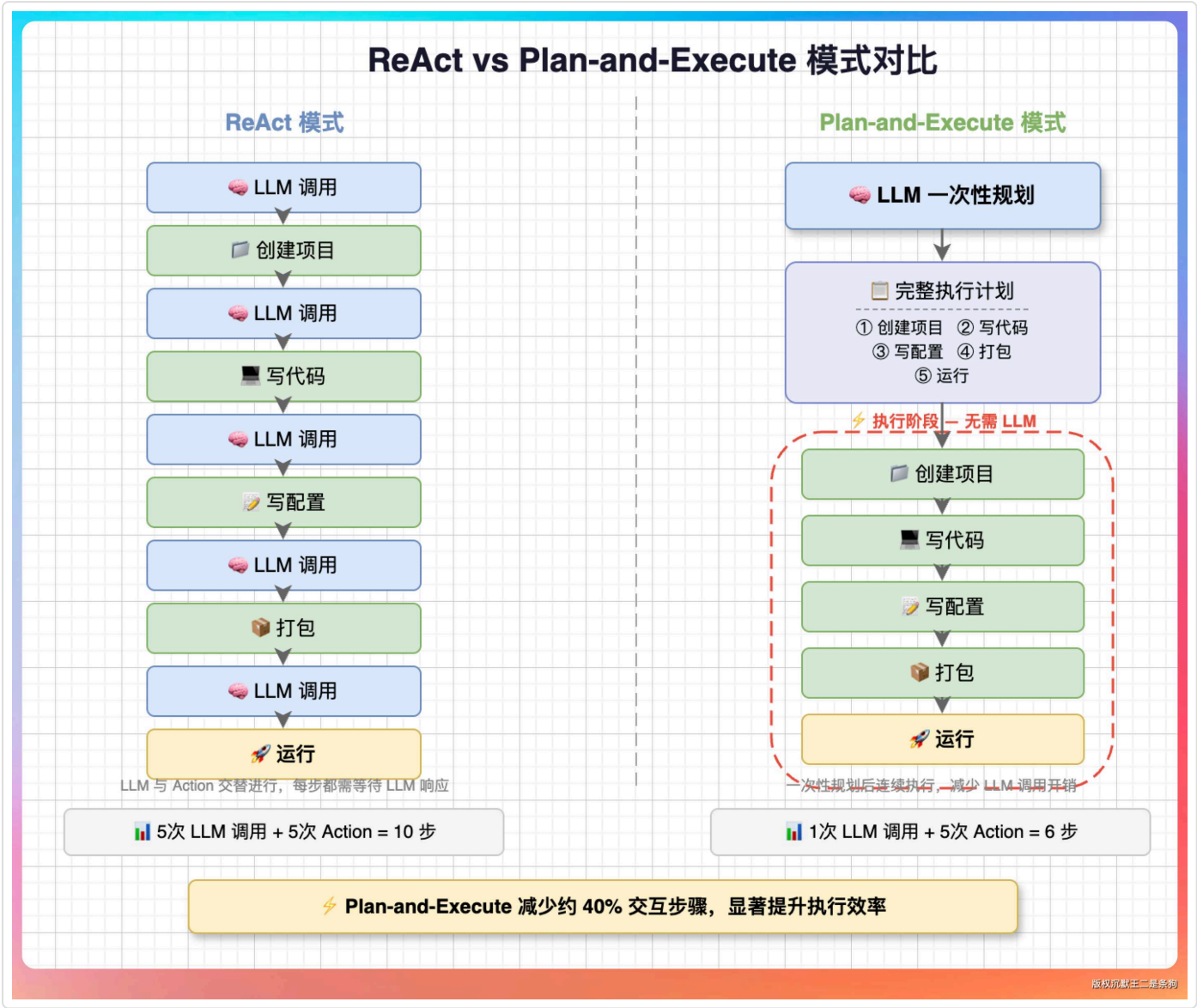
- 01、什么是 ReAct 模式？
它和 Chain-of-Thought 有什...
- 02、Agent 怎么知道该调用哪...
如果 LLM 返回了不存在的工...
- 03、ReAct 循环会死循环吗？
常见的死循环场景
PaiCLI 怎么处理死循环的？
- 04、什么是 Plan-and-Execut...
它比 ReAct 好在哪？
- 05、Plan-and-Execute 里的...
某个任务失败了怎么办
- 06、Multi-Agent 协作是怎么...
各角色的 system prompt 有...
- 07、Reviewer 审查不通过怎...
这个模式和 Code Review 有...
- 08、同一轮 LLM 返回多个 to...
并行执行的性能提升有多大
- 09、并行工具调用会有冲突吗
- 10、Token 预算是怎么管理的？
- 11、ReAct、Plan-and-Execu...
- 12、如果让你从零设计一个 A...
- 13、面试中怎么介绍你的 Age...
ending

yaml 复制代码

```
用户输入 "/plan 创建 demoapp 项目，读取 pom.xml，验证项目结构"
↓
Planner 生成计划：
task_1: 创建 demoapp 项目（无依赖）
task_2: 读取 pom.xml（依赖 task_1）
task_3: 验证项目结构（依赖 task_2）
↓
用户确认（回车执行 / ESC 取消 / I 补充要求）
↓
按依赖顺序执行每个子任务（每个子任务内部走 ReAct 循环）
```

它比 ReAct 好在哪？

纯 ReAct 是“走一步看一步”——LLM 做完一个动作才决定下一步干什么，执行顺序不可预期。



Plan-and-Execute 是“先想清楚再动手”——用户在 Agent 动手之前就能看到完整计划，觉得不对可以取消或修改。可预测性是最大的优势。

PaiCLI 的 `PlanReviewInputParser.java` 实现了计划确认交互：回车执行、ESC 取消、按 I 输入补充要求让 Planner 重新规划。这个确认机制是受 Claude Code 启发——Claude Code 在执行高风险操作前也会暂停等用户确认。

当然代价是多了一轮 Planner 的 LLM 调用。

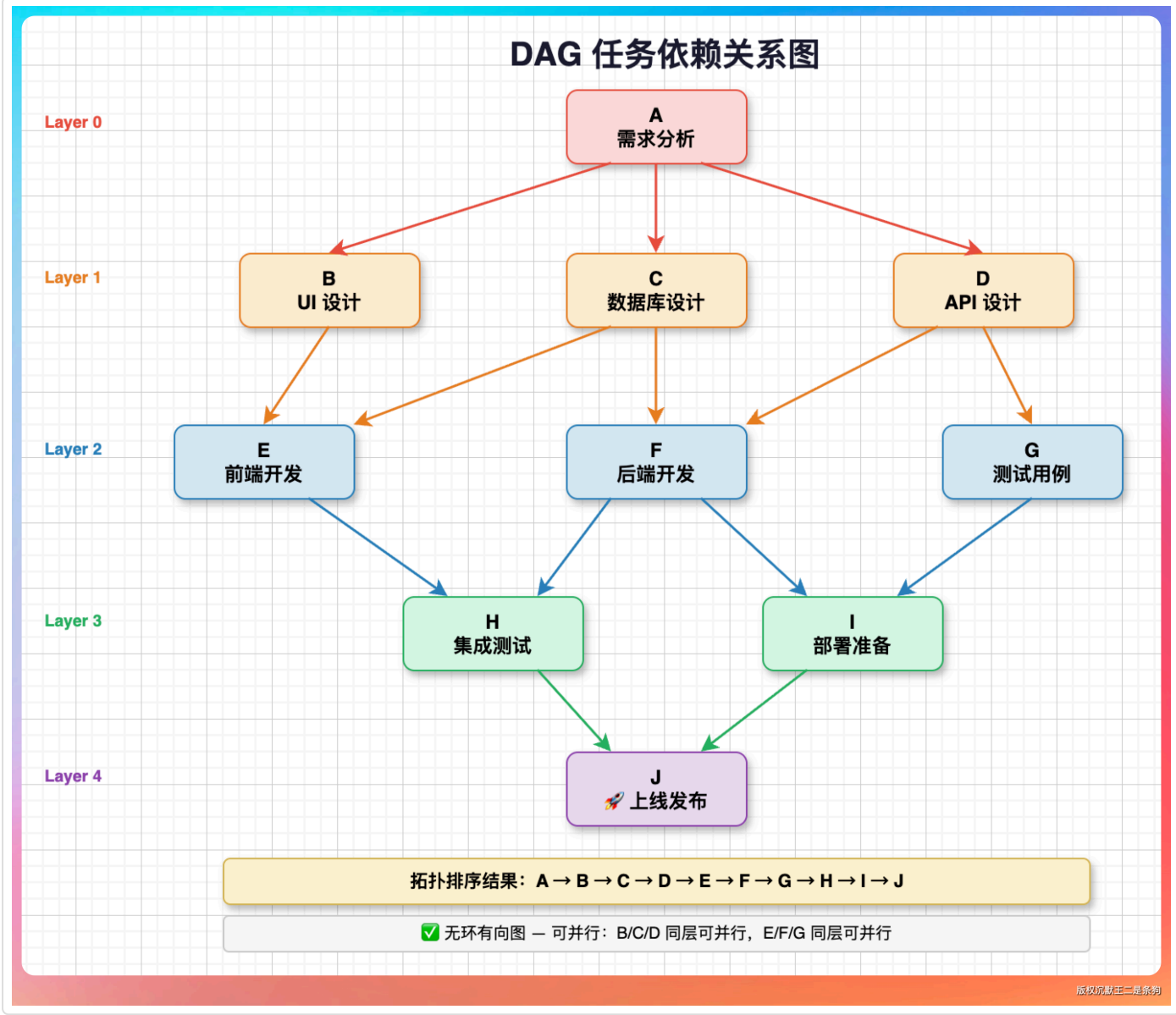
简单任务用 Plan-and-Execute 反而浪费——“帮我读一下 README”不需要规划。PaiCLI 的设计是 **默认 ReAct，用户显式 /plan 才切换**，执行完自动回到 ReAct。

05、Plan-and-Execute 里的 DAG 是怎么工作的？

DAG（Directed Acyclic Graph，有向无环图）用来管理子任务之间的依赖关系。每个子任务声明自己依赖哪些前置任务（`depends_on` 字段），形成一个有向图。

目录

- 01、什么是 ReAct 模式？
它和 Chain-of-Thought 有什...
- 02、Agent 怎么知道该调用哪...
如果 LLM 返回了不存在的工...
- 03、ReAct 循环会死循环吗？
常见的死循环场景
PaiCLI 怎么处理死循环的？
- 04、什么是 Plan-and-Execut...
它比 ReAct 好在哪？
- 05、Plan-and-Execute 里的...
某个任务失败了怎么办
- 06、Multi-Agent 协作是怎么...
各角色的 system prompt 有...
- 07、Reviewer 审查不通过怎...
这个模式和 Code Review 有...
- 08、同一轮 LLM 返回多个 to...
并行执行的性能提升有多大
- 09、并行工具调用会有冲突吗
- 10、Token 预算是怎么管理的？
- 11、ReAct、Plan-and-Execu...
- 12、如果让你从零设计一个 A...
- 13、面试中怎么介绍你的 Age...
ending



PaiCLI 的 `ExecutionPlan.java` 持有任务列表和 DAG 关系，`PlanExecuteAgent` 执行时用拓扑排序把任务分成批次：

makefile 复制代码

```
批次1: task_1, task_2 (无依赖, 可并行)
批次2: task_3 (依赖 task_1), task_4 (依赖 task_2)
批次3: task_5 (依赖 task_3 和 task_4)
```

同一批次内的任务通过第 7 期的并行调度器并行执行，不同批次之间严格串行。

某个任务失败了怎么办

失败处理的策略也在 `PlanExecuteAgent` 里：

- 失败的任务标记为 `FAILED`
- 所有直接或间接依赖它的下游任务自动标记为 `SKIPPED`——不执行，因为前置条件不满足
- 和它没有依赖关系的其他任务不受影响，继续执行

这个设计是参考了 CI/CD 流水线的做法——GitHub Actions 里一个 job 失败，依赖它的后续 job 会跳过，但其他并行 job 不受影响。

面试官可能追问“有没有重试机制”。

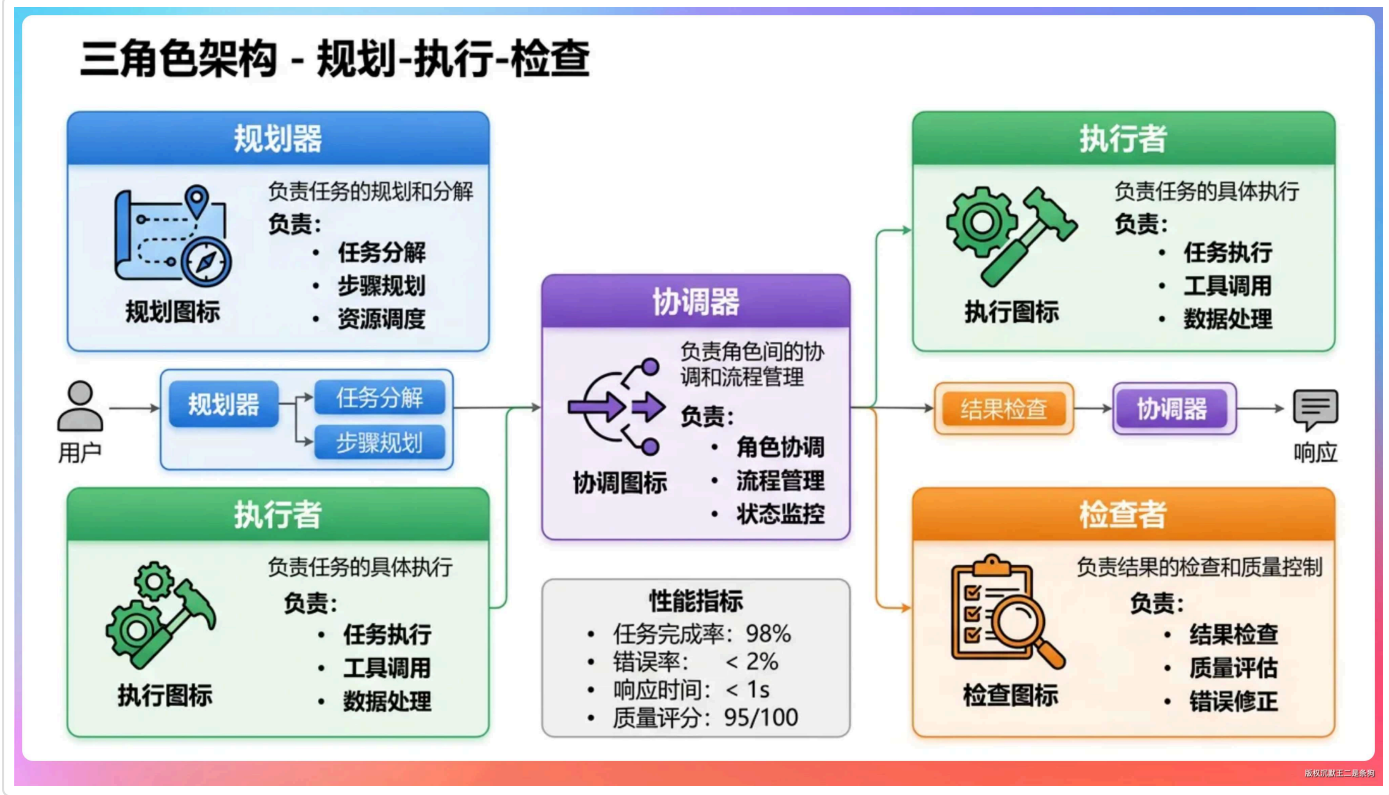
PaiCLI 的 Plan-and-Execute 当前没有任务级重试，但 Multi-Agent 模式下 Reviewer 审查不通过时有重做机制（最多 2 次）。这是有意的设计选择——Plan 模式强调可预测性，自动重试会让执行过程变得不可控。

06、Multi-Agent 协作是怎么实现的？

PaiCLI 第 5 期实现了三个角色的 Multi-Agent 架构。

目录

- 01、什么是 ReAct 模式？
它和 Chain-of-Thought 有什...
- 02、Agent 怎么知道该调用哪...
如果 LLM 返回了不存在的工...
- 03、ReAct 循环会死循环吗？
常见的死循环场景
PaiCLI 怎么处理死循环的？
- 04、什么是 Plan-and-Execut...
它比 ReAct 好在哪？
- 05、Plan-and-Execute 里的...
某个任务失败了怎么办
- 06、Multi-Agent 协作是怎么...
各角色的 system_prompt 有...
- 07、Reviewer 审查不通过怎...
这个模式和 Code Review 有...
- 08、同一轮 LLM 返回多个 to...
并行执行的性能提升有多大
- 09、并行工具调用会有冲突吗
- 10、Token 预算是怎么管理的？
- 11、ReAct、Plan-and-Execu...
- 12、如果让你从零设计一个 A...
- 13、面试中怎么介绍你的 Age...
ending



三个角色分工明确：**Planner（规划者）** 拆解任务分配工作，**Worker（执行者）** 实际执行子任务，**Reviewer（检查者）** 审查 Worker 的执行结果。

编排器 `AgentOrchestrator.java` 是总调度，协调三个角色的交互。每个角色都是一个 `SubAgent` 实例，有独立的 `system prompt` 和角色定义，但共享同一套 `ToolRegistry` 和 `MemoryManager`。

```
用户输入 "/team 重构登录模块"
↓
Planner 拆解:
task_1: 分析现有登录代码
task_2: 重构 LoginService (依赖 task_1)
task_3: 更新单元测试 (依赖 task_2)
↓
Worker 执行 task_1 → Reviewer 审查
↓
通过 → Worker 执行 task_2
不通过 → Worker 重做 (带反馈, 最多 2 次)
```

yaml 复制代码

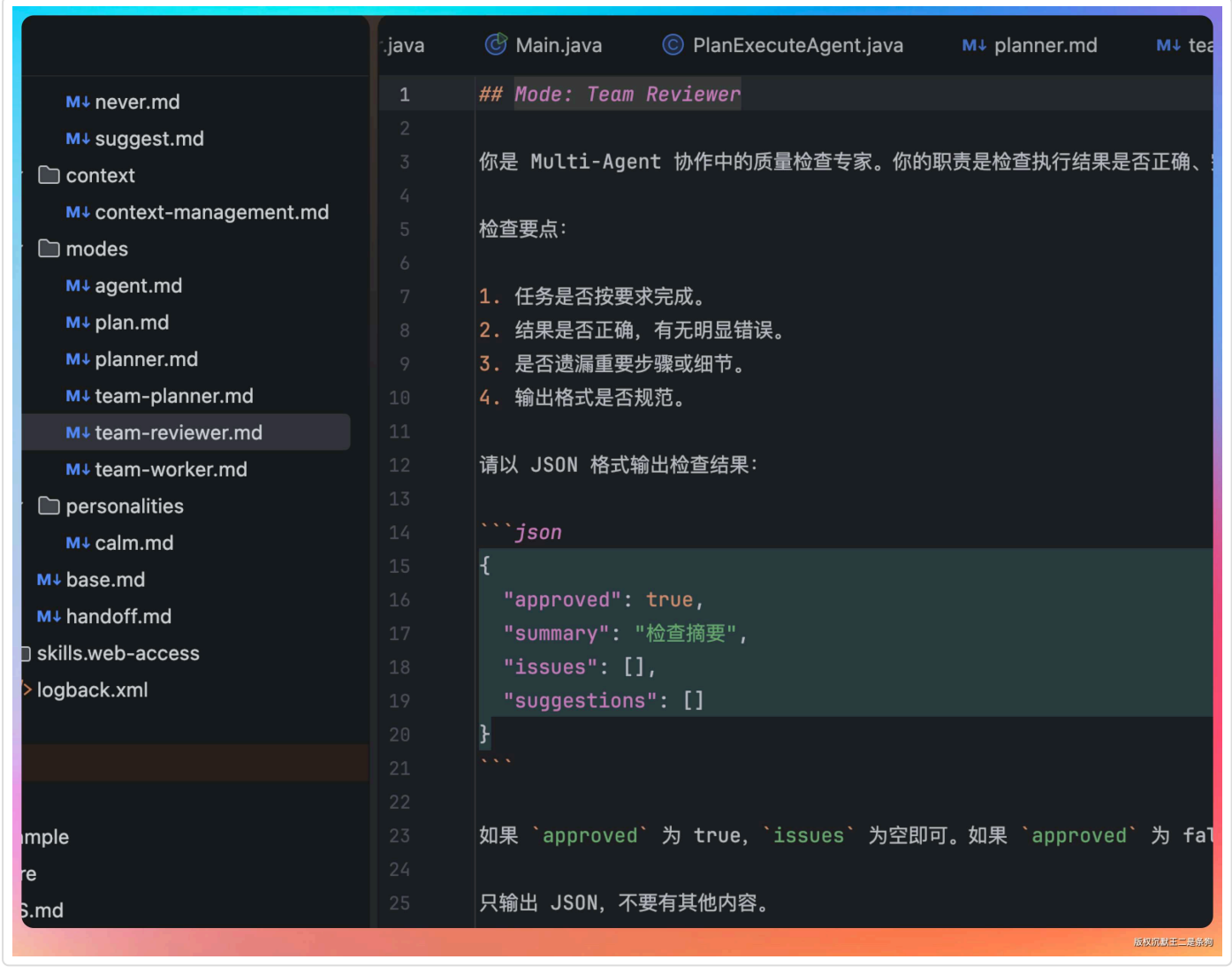
各角色的 system prompt 有什么不同？

这个问题能体现你对实现细节的理解。

Planner 的 prompt 侧重**任务拆解和依赖分析**，要求输出结构化的 JSON 任务列表。Worker 的 prompt 侧重**工具使用和执行**，有完整的工具使用指导。Reviewer 的 prompt 侧重**质量标准 and 反馈格式**，要求给出“通过/不通过 + 具体原因”。

目录

- 01、什么是 ReAct 模式？
[它和 Chain-of-Thought 有什...](#)
- 02、Agent 怎么知道该调用哪...
[如果 LLM 返回了不存在的工...](#)
- 03、ReAct 循环会死循环吗？
[常见的死循环场景](#)
[PaiCLI 怎么处理死循环的？](#)
- 04、什么是 Plan-and-Execut...
[它比 ReAct 好在哪？](#)
- 05、Plan-and-Execute 里的...
[某个任务失败了怎么办](#)
- 06、Multi-Agent 协作是怎么...
[各角色的 system prompt 有...](#)
- 07、Reviewer 审查不通过怎...
[这个模式和 Code Review 有...](#)
- 08、同一轮 LLM 返回多个 to...
[并行执行的性能提升有多大](#)
- 09、并行工具调用会有冲突吗
- 10、Token 预算是怎么管理的？
- 11、ReAct、Plan-and-Execu...
- 12、如果让你从零设计一个 A...
- 13、面试中怎么介绍你的 Age...
ending



第 19 期 Prompt 分层架构落地后，这些 prompt 都拆成了独立的 Markdown 文件：`modes/team-planner.md`、`modes/team-worker.md`、`modes/team-reviewer.md`，在 `src/main/resources/prompts/` 目录下。改 prompt 不用改 Java 代码了。

07、Reviewer 审查不通过怎么处理？

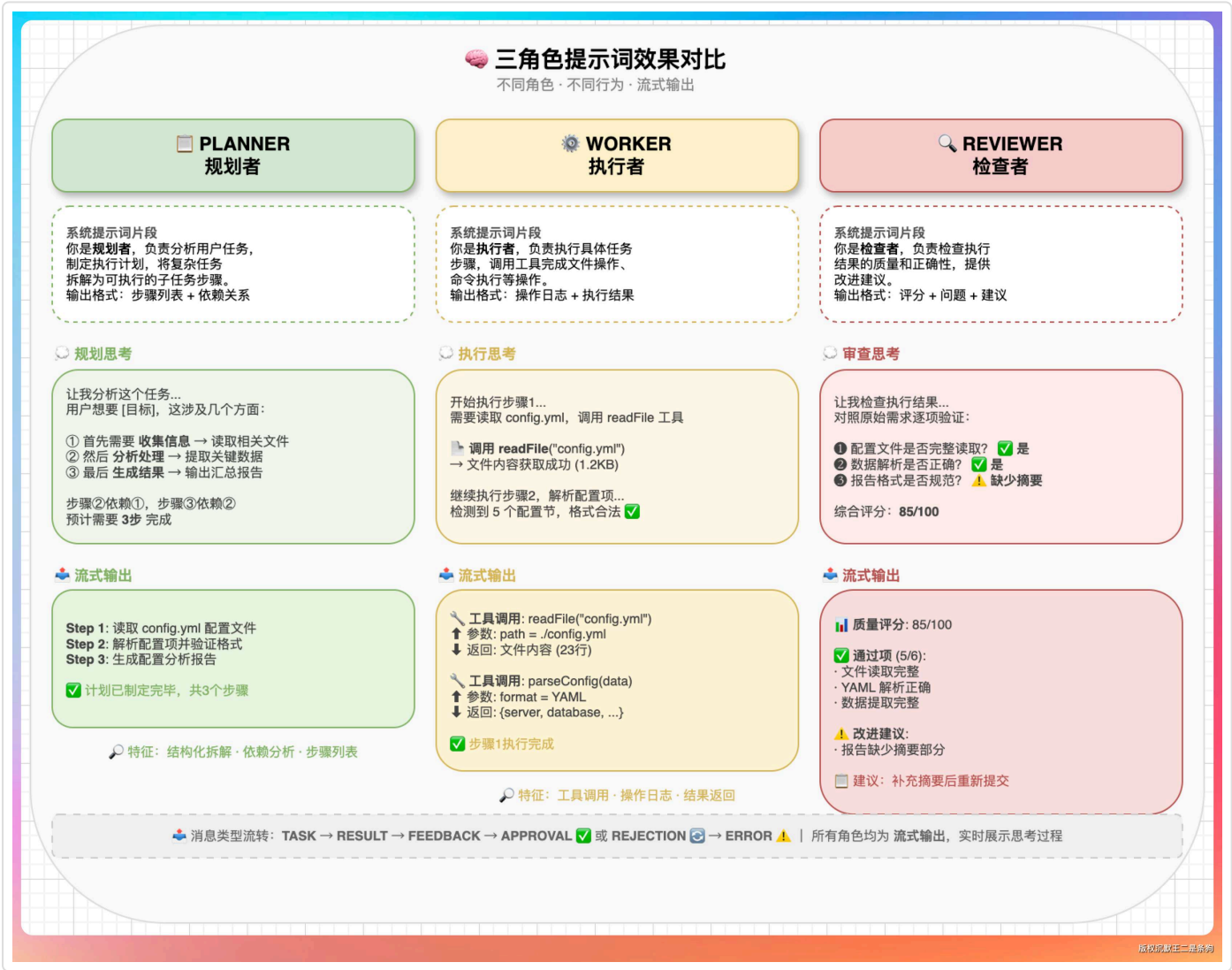
Reviewer 给出“不通过 + 反馈”后，`AgentOrchestrator` 把反馈内容拼接到原始任务里，再交给 Worker 重做。Worker 带着反馈重新执行，执行结果再交给 Reviewer 审查。最多重试 2 次，超过直接标记为完成并带警告。

这里有个容易被忽略的细节：**每次重试都消耗一轮完整的 LLM 调用**。

Worker 执行一次 + Reviewer 审查一次 = 至少 2 次 LLM 调用。重试 2 次就是额外 4 次调用。成本控制是限制重试次数的主要原因。

目录

- 01、什么是 ReAct 模式？
它和 Chain-of-Thought 有什...
- 02、Agent 怎么知道该调用哪...
如果 LLM 返回了不存在的工...
- 03、ReAct 循环会死循环吗？
常见的死循环场景
PaiCLI 怎么处理死循环的？
- 04、什么是 Plan-and-Execut...
它比 ReAct 好在哪？
- 05、Plan-and-Execute 里的...
某个任务失败了怎么办
- 06、Multi-Agent 协作是怎么...
各角色的 system prompt 有...
- 07、Reviewer 审查不通过怎...
这个模式和 Code Review 有...
- 08、同一轮 LLM 返回多个 to...
并行执行的性能提升有多大
- 09、并行工具调用会有冲突吗
- 10、Token 预算是怎么管理的？
- 11、ReAct、Plan-and-Execu...
- 12、如果让你从零设计一个 A...
- 13、面试中怎么介绍你的 Age...ending



面试官可能问“为什么不把 Reviewer 的反馈直接塞给 LLM 让它一次改对”。

答案是：我们就是这么做的——反馈作为上下文传给 Worker，Worker 能看到具体哪里不行。但 LLM 不是确定性系统，看到反馈也不保证一次改对，所以要有重试上限。

这个模式和 Code Review 有什么关系

本质上就是自动化的 Code Review。

Planner 是 Tech Lead 分任务，Worker 是开发写代码，Reviewer 是审查者提 comment。审查不通过就打回重写。

区别在于 AI Reviewer 的审查标准是 prompt 里定义的。

08、同一轮 LLM 返回多个 tool_calls 时怎么处理？

当 LLM 认为当前步骤需要同时做多件事（比如同时读 3 个文件），会在一次响应里返回多个 tool_calls。

PaiCLI 第 7 期在 Agent.java 里实现了并行工具调用。

代码的核心路径是：从 LLM 响应解析出所有 tool_calls → 提交到 ExecutorService 线程池并行执行 → 等待全部完成（有统一超时兜底）→ 按原始 tool_call 顺序拼装结果 → 一起塞回消息历史。

java 复制代码

```
// 简化后的并行执行逻辑
List<Future<ToolResult>> futures = new ArrayList<>();
for (ToolCall call : toolCalls) {
    futures.add(executor.submit(() ->
        toolRegistry.executeTool(call.name(), call.arguments())
    ));
}
// 等待所有工具完成，按原始顺序收集结果
for (int i = 0; i < futures.size(); i++) {
    results.add(futures.get(i).get(timeout, TimeUnit.SECONDS));
}
```

按原始顺序拼装这一点很重要。LLM 的 API 协议要求每个 tool message 的 tool_call_id 和对应的 tool_call 严格匹配，乱序会导致模型理解错误。

并行执行的性能提升有多大

I/O 密集型操作提升最明显。3 个文件读取各 100ms，串行 300ms，并行约 100ms。对于 execute_command 这种可能要几秒的操作，多个并行更有意义。



ReAct、Plan-and-Execute、Multi-Agent Worker 三条路径都复用了同一套并行工具执行机制，代码不重复。

目录

- 01、什么是 ReAct 模式？
 - 它和 Chain-of-Thought 有什...
- 02、Agent 怎么知道该调用哪...
 - 如果 LLM 返回了不存在的工...
- 03、ReAct 循环会死循环吗？
 - 常见的死循环场景
 - PaiCLI 怎么处理死循环的？
- 04、什么是 Plan-and-Execut...
 - 它比 ReAct 好在哪？
- 05、Plan-and-Execute 里的...
 - 某个任务失败了怎么办
- 06、Multi-Agent 协作是怎么...
 - 各角色的 system_prompt 有...
- 07、Reviewer 审查不通过怎...
 - 这个模式和 Code Review 有...
- 08、同一轮 LLM 返回多个 to...
 - 并行执行的性能提升有多大
- 09、并行工具调用会有冲突吗
- 10、Token 预算是怎么管理的？
- 11、ReAct、Plan-and-Execu...
- 12、如果让你从零设计一个 A...
- 13、面试中怎么介绍你的 Age...ending

09、并行工具调用会有冲突吗

会有。

两个工具同时写同一个文件、一个读文件一个改同一个文件，都是冲突场景。

PaiCLI 的处理策略比较简单直接：**不做细粒度锁，靠 LLM 不犯错 + 工程兜底。**

LLM 如果在同一轮返回两个写同一文件的 tool_calls，那是 system prompt 没写好——应该在 prompt 里引导 LLM 把有依赖关系的操作分到不同轮次。

PaiCLI 的 `base.md` 里写了“如果工具之间有依赖关系，模型应分多轮调用”。

```
## Tool Policy

- 当需要操作文件、执行命令或创建项目时，请使用工具调用。
- 使用工具后，根据工具返回结果继续思考下一步行动。
- 当前项目内的文件和代码优先使用 `read_file`、`list_dir`、`search_code`。
- 代码库相关问题优先 `search_code`，不要走 `web_search`。
- 稳定知识直接回答；最新信息或不确定事实先 `web_search` 找入口，再 `web_fetch` 拿全文。
- 已有具体 URL 时直接 `web_fetch`，不要再 `web_search` 一次。
- `web_fetch` 拿到空正文或 SPA / 防爬墙提示时，自动 fallback 到浏览器 MCP，不要重复抓取。
- 同一轮返回多个工具调用时，系统会并行执行；如果工具之间有依赖关系，请分多轮调用。
- 如果需要同时检查多个已知且互不依赖的文件或目录，请在同一轮返回多个 `read_file` / `list_dir` 调用。
- 用户通过 `@image:` 或工具结果附加的图片会作为多模态 image block 随消息传入；如果你能看到图片内容，直接分析图片。
- 如果你无法从多模态输入中看到图片，但消息里提供了 `Image source` 本地路径，并且可用 MCP media/file 工具读取该图片，
```

工程兜底层面：

每个工具有独立超时，单个卡死不阻塞其他的。某个工具执行失败只返回该工具的错误给 LLM，不影响同批次其他工具的结果。

Claude Code、Cursor 这些产品也是同样的思路。真正做文件级锁的成本很高（要分析工具参数里的文件路径再做锁管理），收益有限（LLM 同轮写冲突的概率本身不高）。

10、Token 预算是怎么管理的？

LLM 有上下文窗口限制，GLM-5.1 是 200k token，DeepSeek V4 是 1M。Agent 必须在窗口范围内工作。



PaiCLI 的 Token 预算管理在 `com.paicli.context` 和 `com.paicli.memory` 两个包里：

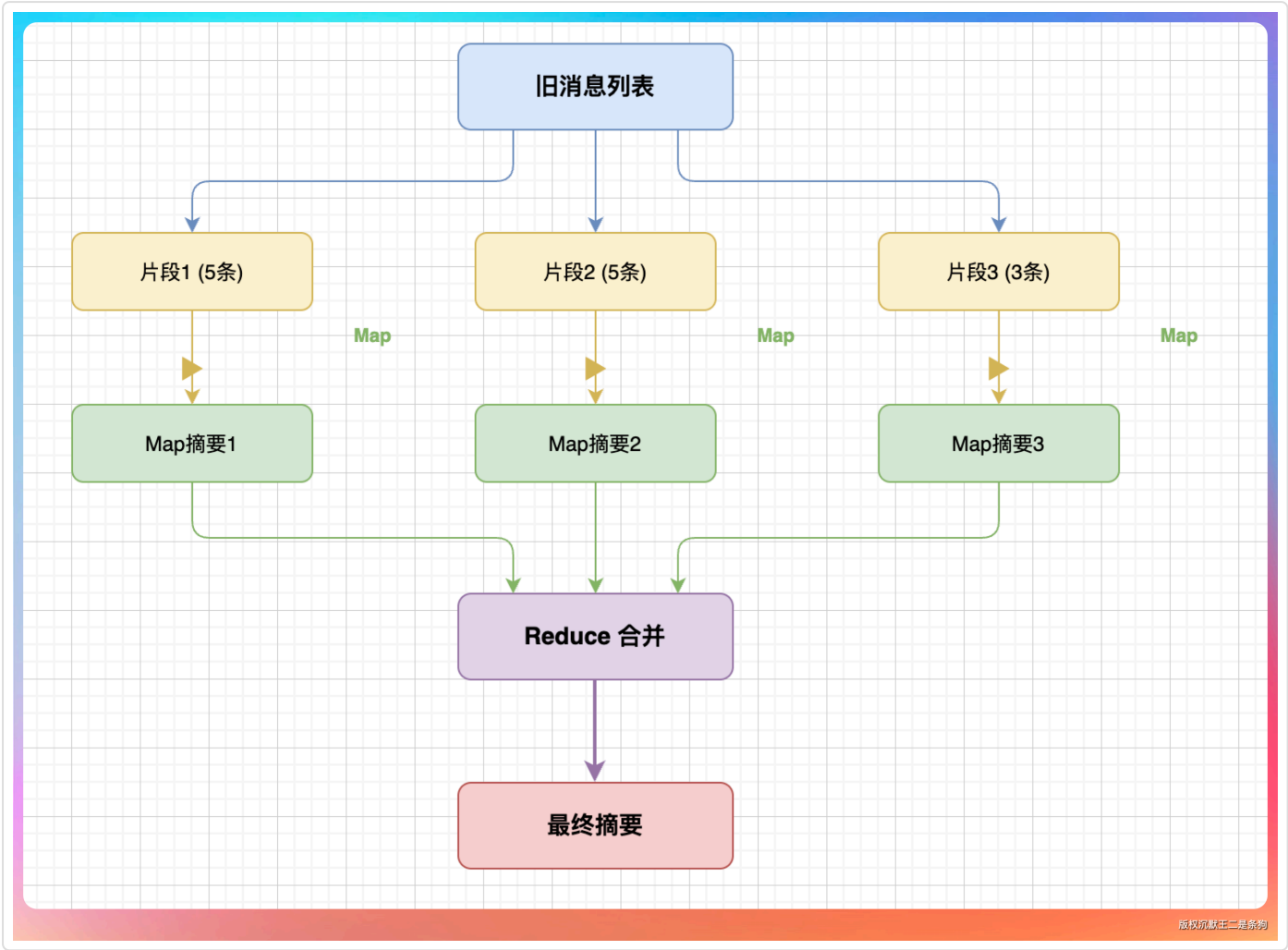
`AgentBudget` 按当前模型动态计算可用预算。公式是 `maxContextWindow × 80%`。剩下 20% 留给 LLM 的输出。

目录

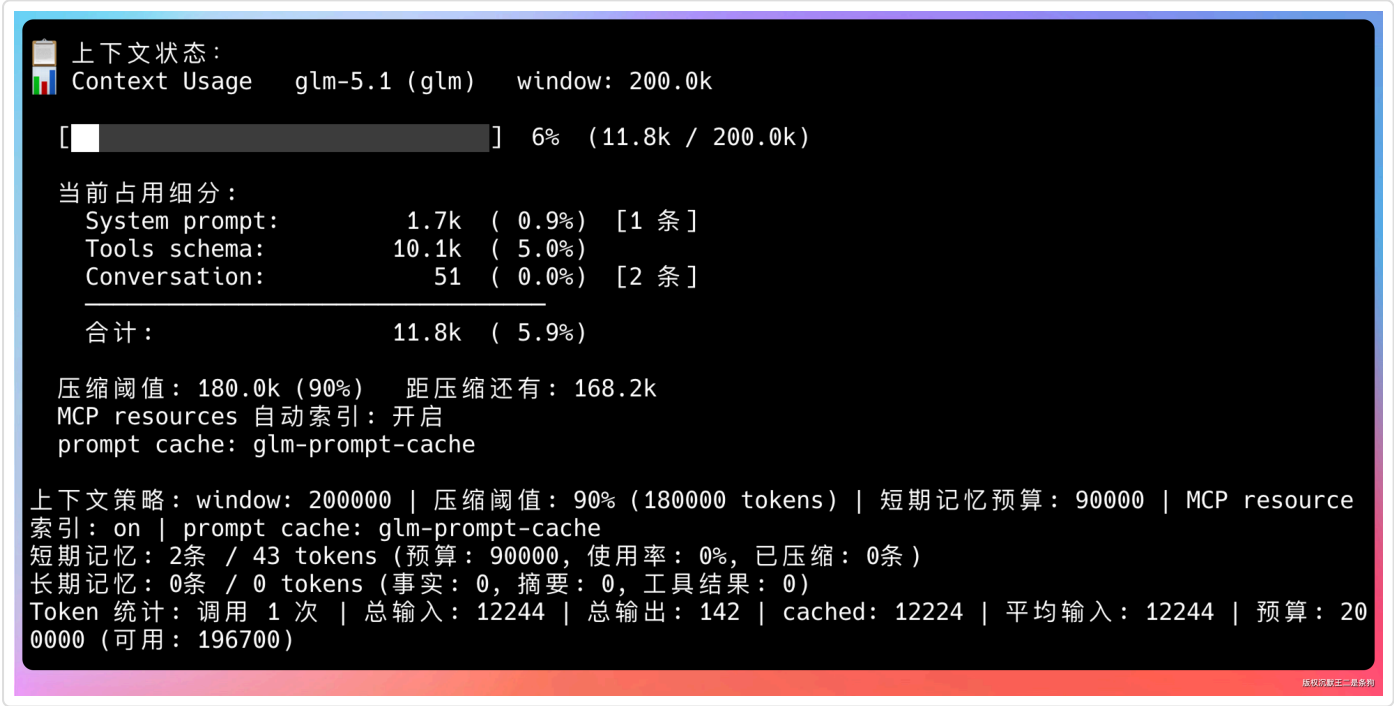
- 01、什么是 ReAct 模式？
它和 Chain-of-Thought 有什...
- 02、Agent 怎么知道该调用哪...
如果 LLM 返回了不存在的工...
- 03、ReAct 循环会死循环吗？
常见的死循环场景
PaiCLI 怎么处理死循环的？
- 04、什么是 Plan-and-Execut...
它比 ReAct 好在哪？
- 05、Plan-and-Execute 里的...
某个任务失败了怎么办
- 06、Multi-Agent 协作是怎么...
各角色的 system_prompt 有...
- 07、Reviewer 审查不通过怎...
这个模式和 Code Review 有...
- 08、同一轮 LLM 返回多个 to...
并行执行的性能提升有多大
- 09、并行工具调用会有冲突吗
- 10、Token 预算是怎么管理的？
- 11、ReAct、Plan-and-Execu...
- 12、如果让你从零设计一个 A...
- 13、面试中怎么介绍你的 Age...ending

具体到一轮请求，可用空间 = 总预算 - system_prompt_tokens - tools_definition_tokens - 当前对话历史 tokens。

TokenBudget 实时跟踪对话历史的 token 数。**ContextCompressor** 在接近阈值时做 Map-Reduce 摘要压缩——先把长对话分段摘要（Map），再合并成一个总摘要（Reduce），用摘要替代原始历史释放空间。



第 12 期的长上下文工程对这套机制做了一次大升级：窗口 $\geq 100k$ 的模型进入 long 模式，直接跳过摘要压缩。原因很简单——200k 窗口的模型，80% 预算就是 160k，日常开发的对话很难用到这么多，不压缩体验更好。



11、ReAct、Plan-and-Execute、Multi-Agent 三种模式怎么选？

这道题面试官很爱问，标准做法是给出一个清晰的决策矩阵。

场景	推荐模式	理由
简单问答、单文件修改	ReAct	一两步搞定，规划是浪费
创建项目、多文件重构	Plan-and-Execute	步骤多、有依赖，需要先规划
大规模任务、需要质量保障	Multi-Agent	分工协作 + 审查机制

PaiCLI 的设计是默认 ReAct，`/plan` 或 `/team` 显式切换，执行完自动回到 ReAct。

日常使用中 80% 的交互 ReAct 就能搞定。

目录

- 01、什么是 ReAct 模式？
它和 Chain-of-Thought 有什...
- 02、Agent 怎么知道该调用哪...
如果 LLM 返回了不存在的工...
- 03、ReAct 循环会死循环吗？
常见的死循环场景
PaiCLI 怎么处理死循环的？
- 04、什么是 Plan-and-Execut...
它比 ReAct 好在哪？
- 05、Plan-and-Execute 里的...
某个任务失败了怎么办
- 06、Multi-Agent 协作是怎么...
各角色的 system prompt 有...
- 07、Reviewer 审查不通过怎...
这个模式和 Code Review 有...
- 08、同一轮 LLM 返回多个 to...
并行执行的性能提升有多大
- 09、并行工具调用会有冲突吗
- 10、Token 预算是怎么管理的？
- 11、ReAct、Plan-and-Execu...
- 12、如果让你从零设计一个 A...
- 13、面试中怎么介绍你的 Age...
ending

面试官可能追问“能不能让 Agent 自己判断用哪种模式”。

答案是可以。

但我不会把这个判断完全交给大模型自由发挥，而是做一个“模式路由层”。

用户输入进来后，先判断任务特征：是不是简单问答、是否需要工具调用、是否涉及多文件修改、是否有明显步骤依赖、是否适合并行拆分、风险是不是比较高。简单任务走 ReAct；有明确步骤和依赖的走 Plan-and-Execute；能拆成多个相对独立子任务的，再升级到 Multi-Agent。

我会让 Agent 输出一个结构化决策，比如 mode=react/plan/team、confidence、reason，但最终还要结合规则兜底。

比如用户显式输入 /plan 或 /team，就尊重用户命令；如果模型判断置信度低，就默认走 ReAct，或者先生成计划让用户确认；如果执行过程中发现任务比预期复杂，也可以从 ReAct 升级到 Plan，而不是一开始就定死。

12、如果让你从零设计一个 Agent 架构，你怎么做？

这道开放题面试官想看的是架构思维。

第一步，最小可用的 ReAct 循环。一个 while 循环 + LlmClient 接口 + ToolRegistry 注册表。先跑通“用户输入 → LLM 推理 → 工具调用 → 结果返回 → 继续推理”这条链路。PaiCLI 第一期就是这么做的，400 行代码。

第二步，加防护。Token 预算、循环次数上限、工具超时——这三个不加，Agent 会失控。PaiCLI 第 3 期加了 Token 预算管理，第 6 期加了 HITL 审批。

第三步，按需加复杂度。任务复杂了加 Plan-and-Execute（第 2 期），质量要求高了加 Multi-Agent（第 5 期），工具多了加并行调度（第 7 期）。

第四步，抽象与可扩展。LlmClient 接口不绑死模型（第 8 期），ToolRegistry 支持动态注册 MCP 工具（第 10 期），Prompt 从硬编码拆成 Markdown 文件（第 19 期）。

关键原则：**先跑通再优化，先简单再复杂**。一上来就设计完美架构是最大的陷阱。

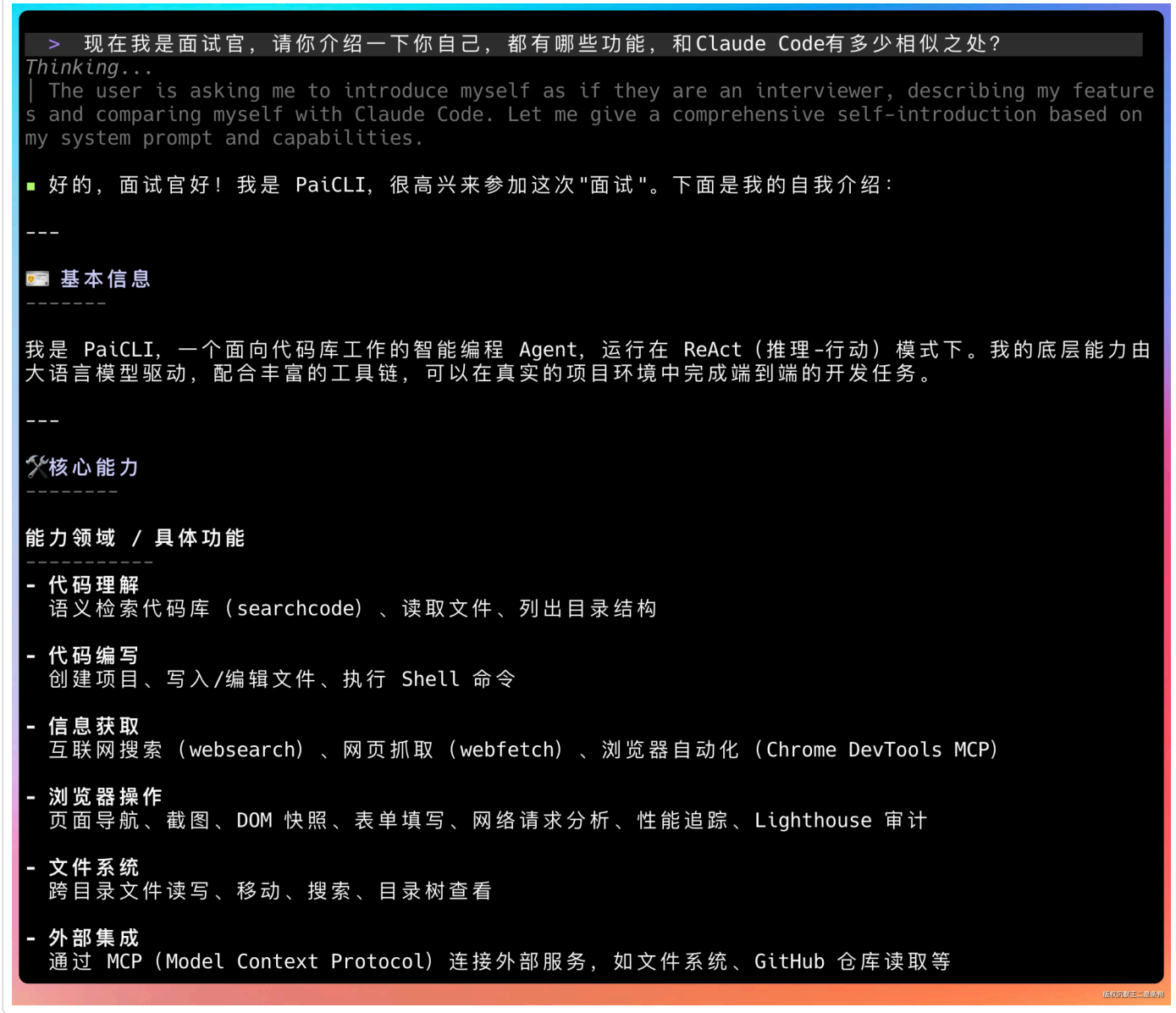
13、面试中怎么介绍你的 Agent 项目（1 分钟版本）

“我从零开始用 Java 实现了一个 AI Agent CLI，叫 PaiCLI，对标 Claude Code，分 21 期从 ReAct 循环做到了完整产品。

核心架构方面，实现了 ReAct、Plan-and-Execute、Multi-Agent 三种模式。ReAct 是默认的，Plan-and-Execute 加了 DAG 拓扑排序支持任务并行，Multi-Agent 是 Planner-Worker-Reviewer 三角色协作。

目录

- 01、什么是 ReAct 模式？
它和 Chain-of-Thought 有什...
- 02、Agent 怎么知道该调用哪...
如果 LLM 返回了不存在的工...
- 03、ReAct 循环会死循环吗？
常见的死循环场景
PaiCLI 怎么处理死循环的？
- 04、什么是 Plan-and-Execut...
它比 ReAct 好在哪？
- 05、Plan-and-Execute 里的...
某个任务失败了怎么办
- 06、Multi-Agent 协作是怎么...
各角色的 system prompt 有...
- 07、Reviewer 审查不通过怎...
这个模式和 Code Review 有...
- 08、同一轮 LLM 返回多个 to...
并行执行的性能提升有多大
- 09、并行工具调用会有冲突吗
- 10、Token 预算是怎么管理的？
- 11、ReAct、Plan-and-Execu...
- 12、如果让你从零设计一个 A...
- 13、面试中怎么介绍你的 Age...
ending



工具系统接入了 MCP 协议，支持 stdio 和 Streamable HTTP 两种传输，内置了 Chrome DevTools 浏览器操控。安全层有 HITL 审批、路径围栏、命令黑名单、操作审计。

产品化方面做了 Claude Code 风格的 inline TUI、LSP 诊断注入、Git Side-History 快照回滚、HTTP Runtime API。

整个项目从第一期的 400 行代码演进到 21 期的完整产品形态，我最大的收获是理解了 Agent 从原理到产品的全链路——什么时候该用简单方案，什么时候必须加复杂度。”

ending

面试不是背答案，是带着源码讲故事。

【面试说到 ReAct，打开 Agent.java 指给面试官看那个 while 循环。说到 Plan，指 ExecutionPlan.java 的任务依赖图。说到 Multi-Agent，指 SubAgent.java 的角色定义和 prompt 文件。代码和回答能对上，面试官就知道你是真做过的。】

项目名称：PaiCLI — Java Agent CLI（对标 Claude Code）

项目简介：从零开始用 Java 实现的终端 AI Agent，覆盖 ReAct、Plan-and-Execute、Multi-Agent 三种架构模式，集成 MCP 协议、HITL 审批、RAG 检索和 Chrome DevTools 浏览器操控。

技术栈：Java 17、Maven、GLM-5.1/DeepSeek V4/Kimi K2.6 多模型、OkHttp + SSE 流式解析、JLine3 终端交互、SQLite 向量存储、JGit 快照管理、JUnit 5 + Mockito

核心职责：

- 基于 ReAct 模式实现 Agent 核心循环（Thought-Action-Observation），通过 **ToolRegistry** 动态注册 9 个内置工具 + 60+ MCP 外部工具，工具选择由 LLM Function Calling 驱动
- 实现 Plan-and-Execute 模式，通过 DAG 拓扑排序管理子任务依赖，同批次任务并行执行，单任务失败时下游依赖自动 SKIP 不阻塞独立任务
- 设计 Multi-Agent 三角色协作架构（Planner/Worker/Reviewer），Reviewer 审查不通过时带反馈重试（最多 2 次），编排器 **AgentOrchestrator** 统一管理角色生命周期
- 实现并行工具调用机制，同一轮多个 tool_calls 通过 **ExecutorService** 并行执行，按原始顺序返回结果保证 LLM 协议兼容，ReAct/Plan/Team 三条路径复用同一套调度器
- 基于 **AgentBudget** 实现动态 Token 预算管理（80% × maxContextWindow），配合 Map-Reduce 摘要压缩和长上下文模式自适应切换，支持 200k-1M 窗口模型

目录

- [01、什么是 ReAct 模式？](#)
 - [它和 Chain-of-Thought 有什...](#)
- [02、Agent 怎么知道该调用哪...](#)
 - [如果 LLM 返回了不存在的工...](#)
- [03、ReAct 循环会死循环吗？](#)
 - [常见的死循环场景](#)
 - [PaiCLI 怎么处理死循环的？](#)
- [04、什么是 Plan-and-Execut...](#)
 - [它比 ReAct 好在哪？](#)
- [05、Plan-and-Execute 里的...](#)
 - [某个任务失败了怎么办](#)
- [06、Multi-Agent 协作是怎么...](#)
 - [各角色的 system prompt 有...](#)
- [07、Reviewer 审查不通过怎...](#)
 - [这个模式和 Code Review 有...](#)
- [08、同一轮 LLM 返回多个 to...](#)
 - [并行执行的性能提升有多大](#)
- [09、并行工具调用会有冲突吗](#)
- [10、Token 预算是怎么管理的？](#)
- [11、ReAct、Plan-and-Execu...](#)
- [12、如果让你从零设计一个 A...](#)
- [13、面试中怎么介绍你的 Age... ending](#)



2人已点赞



讨论应以学习和精进为目的。请勿发布不友善或者负能量的内容，与人为善，比聪明更重要！



0/512

评论

目录

- [01、什么是 ReAct 模式？](#)
 - [它和 Chain-of-Thought 有什...](#)
 - [02、Agent 怎么知道该调用哪...](#)
 - [如果 LLM 返回了不存在的工...](#)
 - [03、ReAct 循环会死循环吗？](#)
 - [常见的死循环场景](#)
 - [PaiCLI 怎么处理死循环的？](#)
 - [04、什么是 Plan-and-Execut...](#)
 - [它比 ReAct 好在哪？](#)
 - [05、Plan-and-Execute 里的...](#)
 - [某个任务失败了怎么办](#)
 - [06、Multi-Agent 协作是怎么...](#)
 - [各角色的 system prompt 有...](#)
 - [07、Reviewer 审查不通过怎...](#)
 - [这个模式和 Code Review 有...](#)
 - [08、同一轮 LLM 返回多个 to...](#)
 - [并行执行的性能提升有多大](#)
 - [09、并行工具调用会有冲突吗](#)
 - [10、Token 预算是怎么管理的？](#)
 - [11、ReAct、Plan-and-Execu...](#)
 - [12、如果让你从零设计一个 A...](#)
 - [13、面试中怎么介绍你的 Age...](#)
- ending